深圳证券交易所
SHENZHEN
STOCK EXCHANGE

工程技术标准

深圳证券交易所组播行情数据分发协议

(Ver1.01)

深圳证券交易所

二〇二四年八月

文档说明

修订历史		
日期	版本	修订说明
2020-03	0.1	创建
2020-05	0.2	数据类型: 增加变长字符串类型和变长二进制类型 组播报文: 组播报文校验和算法调整为Adler32 组播报文支持管理报文和应用报文
2020-08	1.00	正式稿
2024-08	1.01	（本文档橙色修订部分） 1.修改协议扩展机制; 2.增加协议对分片、压缩、加密的支持; 3.附录增加“打包机制”、“分包机制”说明。

名词释义

词汇缩写	含义
MDGW	Market Data Gateway，行情网关
MDDP	Market Data Distribute Protocol，组播行情数据分发协议，行情网关接入方式配置为组播时使用的分发协议
VSS	Vendor Supplied System,供应商提供的行情接收系统，用于接入行情网关接收行情数据

目 录

一、 前言 1
二、 类型定义 1
三、 报文结构 1
3.1 报文头 2
3.2 报文体 5
3.3 报文尾 6
四、 管理报文定义 6
4.1 组播心跳报文 6
4.2 数据流心跳报文 7
4.3 数据流结束报文 7
五、 附录 9
5.1 发送者-组播流-数据流-数据包关系图 9
5.2 应用消息解码 9
5.3 SENDERID 编码机制 10
5.4 SEQNUM 检测机制 10
5.5 数据源故障检测 11
5.6 公告 11
5.7 打包机制 12
5.8 分包机制 12

深圳证券交易所组播行情数据分发协议

一、 前言

深圳证券交易所（以下简称深交所）组播行情数据分发协议（以下简称 MDDP）是一套适用于证券市场行情数据分发的传输控制协议。MDDP 的主要目的在于以更加高效和便捷的方式为市场参与人的多套业务系统提供实时行情分发服务。

本协议主要关注如何使用 MDDP 进行数据传输，应用层消息的相关定义参见深交所行情数据接口规范。

二、 类型定义

MDDP 使用的所有整数类型均采用网络字节序（BIG-ENDIAN）进行传输。

类型	描述
Int8	8 位有符号整数
Int16	16 位有符号整数
Int32	32 位有符号整数
Int64	64 位有符号整数
uInt8	8 位无符号整数
uInt16	16 位无符号整数
uInt32	32 位无符号整数
uInt64	64 位无符号整数
Char[x]	Char[x]表示字符串，x 表示字符串最大字节数，字符串实际长度小于字段类型最大长度的右补空格；字符串统一使用 UTF-8 编码。
VarChar	变长字符串，长度由前导的 VarCharLength 指定；字符串统一使用 UTF-8 编码。
VarCharLength	变长字符串长度，uInt32
RawData	变长二进制数据，长度由前导的 RawDataLength 指定
RawDataLength	变长二进制数据长度，uInt32
Padding[x]	无格式数据，填充位

三、 报文结构

本章主要描述 MDDP 协议的组播报文定义。每个 MDDP 组播报文都由报文头、报

文体、报文尾三部分构成：

报文头	报文体	报文尾

3.1 报文头

报文头不加密传输，格式如下：

域名	类型	长度（byte）	意义
Protocol	uInt8	1	协议标识，固定为 0xFF
Version	uInt8	1	版本号，当前为 0x01
HeaderSize	uInt8	1	整个报文头的长度，指示报文头由几个 4Byte words 组成。
SenderId	uInt8	1	发送源标识
MarketId	uInt16	2	市场标识
Channel	uInt16	2	频道号，同一市场内的频道不重复，同一频道内的数据内容有序。
SeqNum	Int64	8	报文序列号
MsgCount	uInt16	2	报文内消息条目数
Flag	uInt16	2	报文标志位
TotalFragments	uInt16	2	分片总数（可选）
FragmentNo	uInt16	2	分片序号（可选）
EncodeChecksum	uInt32	4	编码校验和（可选），编码前数据的校验和
Flagx	uInt16	2	扩展标志位 x（可选），x 为第 N 次扩展
Padding	Byte[x]	x	字节对齐填充位（可选），确保报文头长度为 4 字节整数倍，无意义。

1. HeaderSize

整个报文头的长度，长度计算从报文头的第一个字段 Protocol 开始到最后一个字段 Padding 结束(如 HeaderSize=5 表示报文头总长度为 5*4Byte=20Bytes)。

2. SenderId

发送源标识，用于识别同一个组播地址及端口上的不同发送者。

3. MarketId

市场标识，预留字段，目前固定填 1.

4. Channel

行情频道，行情数据可根据业务（如按证券集）分为不同的类别，每个类别的行情数据则可根据其数据量大小进一步分为多个频道。

5. SeqNum

报文序号，每个发送者会在对应市场的 Channel 组合内维持一个从 1 开始递增的序列号，报文内的每一条消息都会让序列号递增 1，下一个报文的 SeqNum 为上一个报文最后一笔消息的 SeqNum+1。根据 ResendBySeqNum 标志位的不同，SeqNum 有两种编码机制：

1. ResendBySeqNum=0，发送者启动或重启后的首个报文从 1 开始编号。
2. ResendBySeqNum=1，发送者使用报文内第一条消息的序列号作为报文序号。（应用消息存在序号并连续递增时才可以使用该模式，如逐笔行情）

6. MsgCount

该字段指示报文体中消息条目数。

7. Flag

Flag 标志位，用于标识报文属性，左起第一位为 Bit15。

Flag Bit	**意义**
Bit 15	PossDupFlag 标志位，可能重复标志 标识报文中的数据为可能重复的数据，此时 SeqNum 可以小于期望的序列号。数据接收者如果已经接收到该数据则可以直接丢弃。 0：非可能重复报文 1：可能重复的报文
Bit 14-13	PacketType 标志位，报文类型标志 00：管理报文 01：应用报文
Bit 12	ResendBySeqNum 标志位 是否支持按报文的 SeqNum 进行重传 0:不支持 1:支持
Bit 11-10	Compress 标志位，压缩标志

	00:不压缩 01:zlib 压缩 10:预留 11:预留
Bit 9-8	Encryption 标志位，加密标志 00:不加密 01:自定义加密 10:预留 11:预留
Bit 7	MsgHeader 标志位,前导长度标志位 0：前导长度不存在 1：前导长度存在
Bit 6	Fragment 标志位，分片包标志 0:不支持分片包； 1:支持分片，将 MDDP 数据包按照指定包大小分为多个 MDDP 分片包发布
Bit 5	编码校验和标志位，0:不提供校验和；1:提供校验和
Bit 4-1	预留
Bit 0	Flag 扩展位 0:下一个 Flag 扩展位不存在； 1:下一个 Flag 扩展位存在，报文头中会出现下一个扩展 Flag 标志位 Flagx

8. TotalFragments

可选字段。表示同一个 MDDP 数据包被分为几个分片。当支持分片时，报文头包含本字段。

9. FragmentNo

可选字段。从 1 开始计数，表示同一个 MDDP 数据包的第几个分片。当支持分片时，报文头包含本字段。

10. EncodeChecksum

可选字段。校验和算法为 Adler32。当编码校验和标志位设置时，报文头包含本字段。可通过该字段对解码后的报文体内容进行校验，如果校验和与本字段不一致则说明解码异常。

11. Flagx

可选字段。扩展标志位 x,x 可以是 1,2,3...。可通过 Flag 标志位字段的扩展标志位 Bit0 新增一个扩展 Flag 标志位字段 Flagx。Flagx 的扩展标志位 Bit0 同样可以继续扩展另外一组 Flag 标志位 Flagx+1。多个扩展标志位及对应可选字段的排列格式如下：

域名	类型	长度
Flag	uInt16	2
OptionField1	xxx	xx
OptionField2	xxx	xx
Flag1	uInt16	2
OptionField3	xxx	xx
OptionField4	xxx	xx

12. Padding

可选字段。报文头长度不为 4Byte 的整数倍时，包头包含这个字段。

3.2 报文体

报文体由 MDDP 消息头和 MDDP 消息体两部分构成，MDDP 消息头指示了报文中每一条应用消息的长度，MDDP 消息体存储了具体的应用消息。一个 MDDP 报文可以包含多条完整的应用消息，消息不跨报文传输。由于有些协议采用了上下文相关的编码机制，无法进行消息切割，因此 MDDP 消息头为可选项，由报文头中的 MsgHeader 标志位标识。

MsgHeader=1 的 MDDP 报文体格式如下：

域名	类型	长度（byte）	意义
Lengths	uInt32[x]	4*MsgCount	MDDP 消息头，指示了报文中每一条消息的长度
Body	Byte[x]	x 由 Lengths 指定	MDDP 消息体

MDDP 消息头	Len1	Len2	Len3	Len4	Len5
MDDP 消息体	Msg1	Msg2	Msg3	Msg4	Msg5

MsgHeader=0 的 MDDP 报文体格式如下：

MDDP 消息体	Msg1	Msg2	Msg3	Msg4	Msg5

3.3 报文尾

报文尾定义了校验和，不加密传输。校验和的计算范围从报文头开始（包括报文头）一直到报文体结束。校验和算法为 Adler32。

域名	类型	长度（byte）	意义
Checksum	uInt32	4	校验和

四、 管理报文定义

本章主要描述 MDDP 协议使用的管理报文定义。

4.1 组播心跳报文

组播流由组播地址、端口及 SenderId 唯一标识，当组播流上没有任何应用层数据时，发送者会在组播流上周期性的发送组播心跳报文，帮助数据接收者检测组播通道状态。组播心跳报文的发送间隔为 5 秒，数据接收者在 3 个心跳周期内没有收到任何数据时，说明该路组播源故障。

Channel 为 0 时表示该报文为“组播心跳报文”，组播心跳报文没有报文体，收到后可以直接丢弃。报文样例：

域名	类型	取值
Protocol	uInt8	协议标识，固定为 0xFF
Version	uInt8	版本号，当前为 0x01
HeaderSize	**uInt8**	**5**
SenderId	uInt8	发送源标识
MarketId	uInt16	1

Channel	**uInt16**	**固定为 0**
SeqNum	Int64	0
MsgCount	**uInt16**	**0**
Flag	uInt16	高位→低位 0000000000000000
CheckSum	uInt32	校验和

4.2 数据流心跳报文

数据流由 Channel 及 SenderId 唯一标识，当报文的 ResendBySeqNum 标志位为 1 时，发送者会在数据流上周期性的数据流心跳报文，用于告知数据接收者当前数据流上已发送的最后一笔数据消息对应的序号。

本协议将“数据流心跳报文”的 SeqNum 设置为当前数据流上已发送的最后一笔消息的 SeqNum。当数据流上没有发送过任何应用消息时，“数据流心跳报文”的 SeqNum 为 0。

Channel 不为 0 且 MsgCount 为 0 时表示该报文为“数据流心跳报文”，数据流心跳报文没有报文体。报文样例：

域名	类型	取值
Protocol	uInt8	协议标识，固定为 0xFF
Version	uInt8	版本号，当前为 0x01
HeaderSize	**uInt8**	**5**
SenderId	uInt8	发送源标识
MarketId	uInt16	1
Channel	**uInt16**	**2011**
SeqNum	Int64	9999
MsgCount	**uInt16**	**0**
Flag	uInt16	高位→低位 0000000000000000
CheckSum	uInt32	校验和

4.3 数据流结束报文

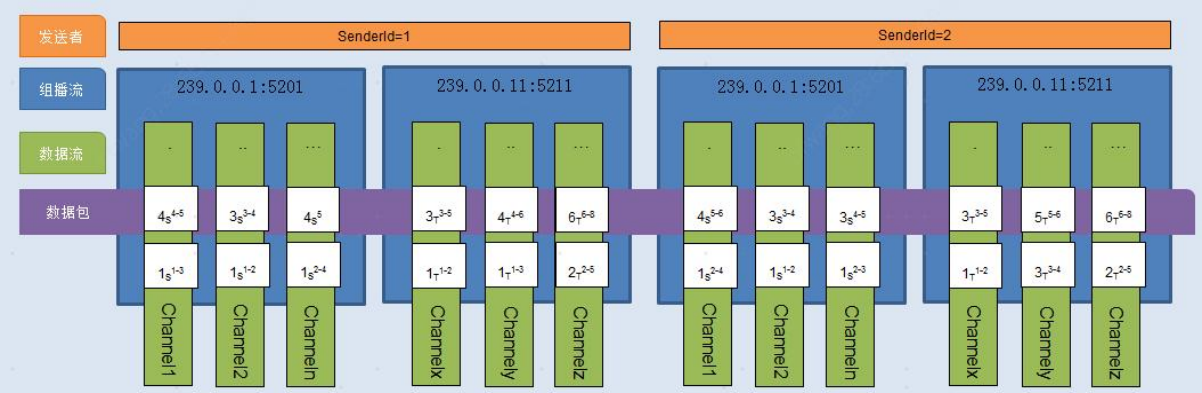
“数据流结束报文”为特殊的数据流心跳报文，用于告知数据接收者当前数据流上所有业务数据已经发送完毕。

Channel 不为 0 且 MsgCount 为 0xFFFF（或写作 65535）时表示该报文为“数据流结束报文”。消息样例：

域名	类型	取值
Protocol	uInt8	协议标识，固定为 0xFF
Version	uInt8	版本号，当前为 0x01
HeaderSize	**uInt8**	**5**
SenderId	uInt8	发送源标识
MarketId	uInt16	1
Channel	**uInt16**	**2011**
SeqNum	Int64	9999
MsgCount	**uInt16**	**0xFFFF**
Flag	uInt16	高位→低位 0000000000000000
CheckSum	uInt32	校验和

五、 附录

5.1 发送者-组播流-数据流-数据包关系图



注 1:Sender1，Sender2 为互为备份的两个发送者

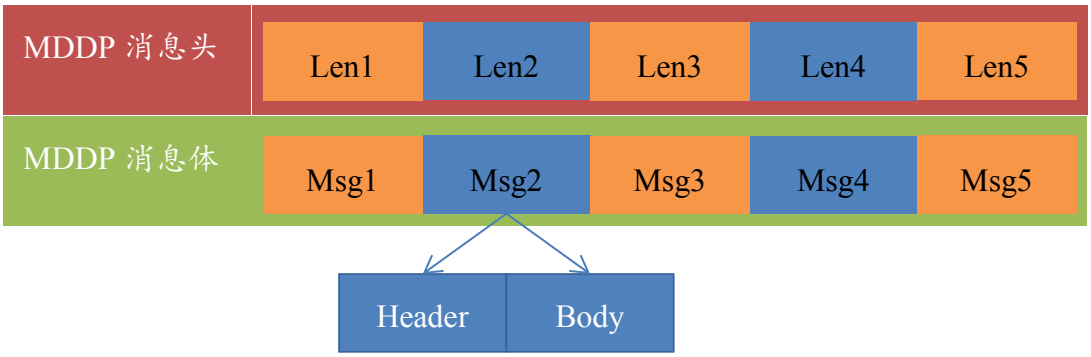
注 2:组播流以组播地址、端口及 Sender 为单位

注 3:数据流以组播流及频道为单位

注 4:N_S^{x-y},N 表示包序号,S 表示快照数据流,x-y 本数据流上的第 x 到第 y 笔快照消息, ResendBySeqNum 标志位为 0

注 5:N_T^{x-y},N 表示包序号,T 表示逐笔数据流,x-y 本数据流上的第 x 到第 y 笔逐笔消息, ResendBySeqNum 标志位为 1

5.2 应用消息解码



MDDP 协议当前仅支持分发深交所二进制行情数据，MDDP 消息体内存储的应用消息定义参见《深圳证券交易所 Binary 行情数据接口规范》。

MDDP 消息头提供了 MDDP 消息体内每一条消息的长度，接收者可以根据 MDDP 消息头提供的消息长度信息，在不解析 MDDP 消息体的情况下，实现消息的切割。

消息	消息偏移量
Msg1	Offset1=0
Msg2	Offset2= Offset1+Len1
Msg3	Offset3= Offset2+Len2
Msg4	Offset4= Offset3+Len3

Msg5	Offset5= Offset4+Len4

为了提升效率，MDDP 取消了《深圳证券交易所 Binary 行情数据接口规范》中每条消息尾部的校验和，通过 MDDP 报文头的校验和对整个报文进行完整性校验。接收者对 MDDP 报文进行完整性校验后，直接通过 MDDP 消息头完成消息切割并解码消息即可，无需对每一条消息再进行单独的完整性校验。

5.3 SenderId 编码机制

SenderId 编码机制如下：

1. 交易日首次启动时，SenderId 设置为 Sender 在集群中的索引
2. 每次重启，SenderId= Sender 在集群中的索引+(启动次数-1)* 集群的 Sender 数量
3. 当 SenderId> 255 时，SenderId 重置为 Sender 在集群中的索引

SenderId 编码机制示例如下：

集群	发送者索引	第 1 次启动	第 2 次启动	第 N 次启动
Sender1	0	**0**	2	0+(N-1)*2
Sender2	1	1	3	1+(N-1)*2

注:如果交易日内重启次数过多或是硬件故障导致重启后 SenderId 无法从磁盘恢复，此时 SenderId 重置为 Sender 在集群中的索引

5.4 SeqNum 检测机制

数据流上每个组播报文的序号连续递增，由于组播会发生乱序、丢包和重包，因此数据接收者应该对数据流上的组播报文序号进行检查。

1. SeqNum<下一个报文期望的报文序号 NextExpectedSeqNum，此时认为数据报文为过期数据直接丢弃。
2. SeqNum=NextExpectedSeqNum，则处理报文，并设置下一个报文期望的报文序号 NextExpectedSeqNum= SeqNum+MsgCount
3. SeqNum>NextExpectedSeqNum，说明可能发生了乱序或丢包，可采用两种处理机制:
   a) 直接判定为网络丢包，从当前数据包开始继续处理后续报文。网络基础设施稳定，很少出现乱序时，可以考虑使用该机制。
   b) 先判定网络发生乱序并缓存数据，当缓存的数据超过预设的缓冲区大小或超时后依然未收到预期的报文，则判定为网络丢包。使用缓存中序号最小报文的作为 NextExpectedSeqNum，继续处理缓存中以及后续的报文。

注 1：接收者的乱序缓冲区需根据网络乱序情况进行设置，如果网络基础设施稳定，很少出现乱序时，则可以设置为一个比较小的值（如 16）

注 2：可重传的数据发生丢包时，可使用 MDGW 提供的应用层重传机制进行恢复

5.5 数据源故障检测

数据源故障重启后，快照类组播报文(ResendBySeqNum 标志位为 0)的 SeqNum 会从 1 开始重新编号，从而导致组播报文的 SeqNum 回退。MDDP 提供两种故障检测机制，便于接收者对数据源故障进行识别并进行恢复。

1、 SenderId 发生变更

当接收者发现数据流上的 SenderId 发生变更时，说明数据源发生重启或是发生了交易日切换，此时应使用最新的数据报文中的消息序号重置下一个期望的组播报文序号。

2、 SenderId 未发生变更，但 SeqNum 回退

该场景一般是因为硬件故障导致 SenderId 无法恢复，从而导致重启后 SenderId 未发生变化但 SeqNum 发生了回退。由于组播可能发生乱序，因此接收者可配置一个阈值，当 SeqNum+阈值< NextExpectedSeqNum 则认为是数据源重启，此时应使用最新的数据报文中的消息序号中重置下一个期望的组播报文序号。否则，则将报文认为是乱序包进行处理。

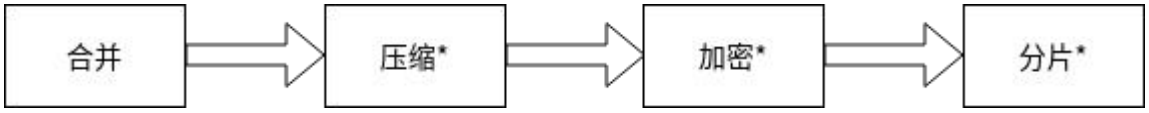
5.6 公告

MDDP 协议支持公告落地功能，可通过 MDGW 提供的公告落地开关启动或关闭。公告落地功能启用后，MDDP 将公告分为公告摘要和公告内容两部分发送，其中公告摘要通过组播报文发送，公告摘要消息定义与二进制数据接口规范中的公告消息一致，公告摘要中不携带公告内容，因此 RawDataLength 被固定置为 0。公告内容则通过文件的方式存储在接收者指定的目录。

5.7 打包机制

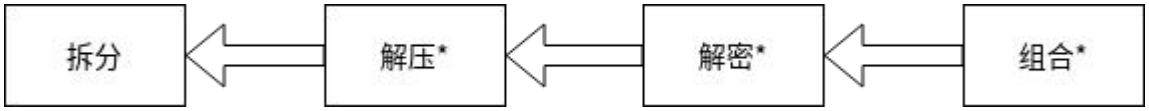
为了提高发送效率，MDDP会将多条消息打包至一个MDDP数据包中发送。MDDP发送端按照合并、压缩、加密及分片的顺序对发送数据做打包处理，MDDP接收端按照组合、解密、解压及拆分的顺序对接收数据做解包处理。

a) 打包流程如下：



1) 将多条消息组装到一个MDDP数据包中。
2) 使用adler32算法计算原始数据包校验和。
3) 如果启用压缩机制，则对原始数据进行压缩处理。
4) 如果启用加密机制，则对上一步骤产生的数据进行加密处理。
5) 如果启用分片机制，则根据分包机制对上一步骤产生的数据进行分包处理。
6) 将原始数据包校验和加入报文头扩展位中，并设置编码校验和标志位。

b) 解包流程如下：

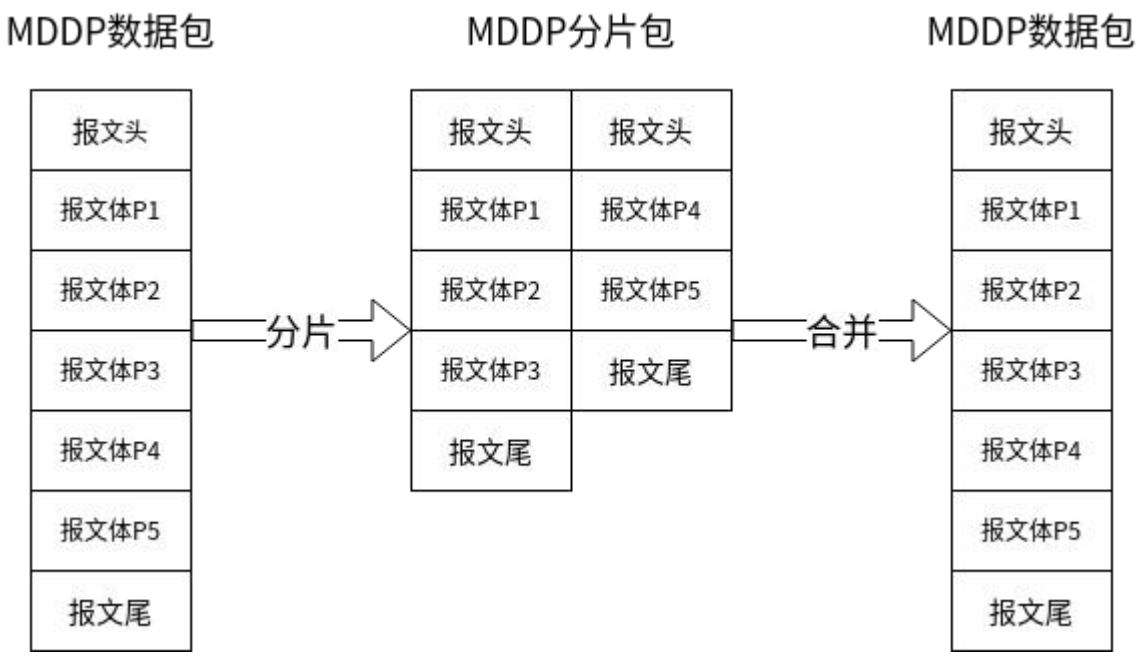


1) 如果启动分片机制，则根据分包机制将多个MDDP分片合并一个MDDP数据包。
2) 如果启用加密机制，则对上一步骤产生的数据进行解密处理。
3) 如果启用压缩机制，则对原始数据进行解压处理。
4) 对上一步中的数据用adler32算法计算校验和与报文头中的编码校验和比较，验证数据的正确性。
5) 根据MDDP报文头或是应用层协议将数据包中的内容拆分成消息。

5.8 分包机制

本文件支持按照指定大小将MDDP数据包切分为多个MDDP分片包。启用分片机制后，MDDP发送方会按照指定的组播包大小对MDDP数据包进行分片处理，分片后每一个分片包大小（包含报文头+报文体+报文尾）不超过指定组播包大小，每一个分片包通过一个组播包进行发送。

启用分片机制后一条消息可能会跨多个分片传输，接收者应将多个分片包组合为一个数据包后再进行解码操作。如果启用了压缩或是加密机制，则同样应将MDDP分片包合并为一个MDDP数据包后再进行解密和解压操作。分包、组包流程如下：



无压缩加密场景下，分包机制示例如下：

a) MDDP数据包报文长度超过指定组播包大小，原数据包内容如下：

域名	类型	取值
Protocol	uInt8	协议标识，固定为 0xFF
Version	uInt8	版本号，当前为 0x01
HeaderSize	uInt8	5
SenderId	uInt8	发送源标识
MarketId	uInt16	1
Channel	uInt16	2011
SeqNum	Int64	1
MsgCount	uInt16	10
Flag	uInt16	高位→低位:xxxxxxxxxx0xxxxx
TotalFragments	uInt16	0
FragmentNo	uInt16	0
报文体	Char[x]	AAAAAAAABBBBBBBBBCCC
CheckSum	uInt32	...

b) 发送者根据指定组播包大小将一个MDDP数据包分成多个MDDP分片包，并为每个MDDP分片包生成校验和。MDDP分片包为Fragment标志位1的数据包，分片包内容如下：

分包1:

域名	类型	取值
Protocol	uInt8	协议标识，固定为 0xFF
Version	uInt8	版本号，当前为 0x01
HeaderSize	uInt8	5
SenderId	uInt8	发送源标识
MarketId	uInt16	1
Channel	uInt16	2011
SeqNum	Int64	1
MsgCount	uInt16	10
Flag	uInt16	高位→低位:xxxxxxxxxx1xxxxx
TotalFragments	uInt16	3
FragmentNo	uInt16	1
报文体	Char[x]	AAAAAAAA
CheckSum	uInt32	...

分包2:

域名	类型	取值
Protocol	uInt8	协议标识，固定为 0xFF
Version	uInt8	版本号，当前为 0x01
HeaderSize	uInt8	5
SenderId	uInt8	发送源标识
MarketId	uInt16	1
Channel	uInt16	2011
SeqNum	Int64	1
MsgCount	uInt16	10
Flag	uInt16	高位→低位:xxxxxxxxxx1xxxxx
TotalFragments	uInt16	3
FragmentNo	uInt16	2
报文体	Char[x]	BBBBBBBBB
CheckSum	uInt32	...

分包3:

域名	类型	取值
Protocol	uInt8	协议标识，固定为 0xFF
Version	uInt8	版本号，当前为 0x01
HeaderSize	uInt8	5
SenderId	uInt8	发送源标识
MarketId	uInt16	1
Channel	uInt16	2011
SeqNum	Int64	1
MsgCount	uInt16	10
Flag	uInt16	高位→低位:xxxxxxxxxx1xxxxx
TotalFragments	uInt16	3
FragmentNo	uInt16	3
报文体	Char[x]	CCC
CheckSum	uInt32	...

c）接收者收到MDDP分片包后，对MDDP分片包的校验和进行校验，校验通过后保存该分片包。当收齐一个MDDP数据包的所有分片包后，应将多个分片包按照顺序组成MDDP数据包，再进行解码操作。